

CLudo

Ludo game in C Language

M.S Thalagala
23002026

SCS 1301 Data Structures and Program Design in C
Dr. Manjusri Wickramasinghe

August 2024

Contents

1	Introduction	3
2	Structure	3
2.1	Board	3
2.2	Players	4
2.3	Pieces	4
3	Functionality	5
3.1	void main()	5
3.2	Start	5
3.3	Game Loop	5
3.4	General Moves	6
3.4.1	Gets another roll	6
3.4.2	baseToStart	6
3.4.3	selectPlayPiece()	6
3.4.4	canCaptureAny()	7
3.4.5	standardMove()	8
3.5	Player Behavior	9
3.5.1	Yellow Player	9
3.5.2	Blue Player	11
3.5.3	Red Player	12
3.5.4	Green Player	13
3.6	Finish	14
4	mistryCell	14
4.1	Generating	14
4.2	Macros	14
4.3	Teleport	14
4.4	Impacts	15
5	Home Move	15
5.1	passApporoch()	15
5.2	homeMove()	15
6	Efficiency	16
6.1	Time	16
6.2	Space	16

1 Introduction

A program to implement Ludo game in C language. Producing Only Command line outputs. This documentation contains the methods to use handle data and problems. There are three .c files includes the full project

1. main.c
2. type.c includes the structure used for store data and macros used.
3. logic.c This file contains all the functions used for implement the whole project.

2 Structure

2.1 Board

The Board has a virtual structures, as per say it works with numbers not as memory cells that holding details. There for space will be minimum without 52 size array.

1. **Standard Path:**
Path has 52 cells from 0 to 51.
2. **Color specific Path:**
The four players have their Own Colour specific path (Home path) to reach HOME. Each colour has 5 cells from 1 to 5
Home is the 6th cell.
3. **Starting Cells and approach Cells:**
Each players Starting Cell - "X" and Approach Cell - "O" defined using Macros

```
#define Y0 0
#define YX 2
#define B0 13
#define BX 15
#define R0 26
#define RX 28
#define G0 39
#define GX 41
```

Justification

Using numbers we can track pieces and details of them easily. But when it comes to compare them we have to go through multiple loops every time.

2.2 Players

Players are defines using structure data type. I used a pointer array of player structure, to keep the each players pointer,By keeping it in a array it is efficient in time to iterate through players.Can access any player $O(1)$ time complexity.

```
player *arrayplayerlist[4] = {&player1, &player2, &player3, &player4}
```

Each player structure holds

```
typedef struct player
{
    short playernum;           // yellow = 0 , blue = 1 , red = 2 , green = 3,
    short inBase;             // Pieces count Base out of 4
    short started;            // Pieces count in Standard Path
    short finished;           // Pieces Count finished the game
    char *color;              // String for Color
    char colorfirst;          // First Letter of Colour
    short x;                  // Starting position of player
    short o;                  // Apporoach cell of player
    short sixrolled;          // count for rolling six
    short *pieceData[4];      // Four size array for hold each piece
}player;
```

int *pieceData[4] is a array of pointers to integer that arrays keeps the pointers to each pieces of itself.

2.3 Pieces

Pieces are defined using short array of size 9. Becaouse of using short data type arrays space will be minimum and accessing to data will be much faster. But when i implementing the code i troubled time to time changing the array size when i need it to expand, so i defined another macro to change array size only using macro size.

```
#define PIECEDATA 9

//pieces array size initiazied
short piecey1[PIECEDATA] = {YELLOW, PIECE1, BASE, 0, 0, 1, 0, 0,-1};
```

There are 16 arrays for 16 players. Each array position holds

- **0 - Player:** Players are numbered from 0 to 3.
- **1 - Piece Number:** Piece numbers are from 0 to 3.
- **2 - Piece Position:** Position can be from 0 to 51.

- **3 - Captures made by Piece:** The count of captured pieces.
- **4 - Moving direction:** 1 for clockwise or -1 for counter-clockwise.
- **5 - Bhawana & Kotuwa impact:** 2*, 0.5*, 0* (if Bhawana).
- **6 - Counter after Impact:** Four rounds Count.
- **7 - Travelled Distance:** The Distance travelled by the piece.
- **8 - Homepath Location:** The location in the home path.

To define array values there are macros

```
#define BASE    -1           #define PIECE1  0
#define YELLOW  0           #define PIECE2  1
#define BLUE    1           #define PIECE3  2
#define RED     2           #define PIECE4  3
#define GREEN   3
```

3 Functionality

3.1 void main()

This is the main function calls only the LUDO() function to start the game.

3.2 Start

1. LUDO()
2. void printbegin() : begining statement
3. int firsthand() :

All players' rolls get stored in firsthandValue[4] array. The rolled values are sorted by merge sort. In this function, while sorting the firsthandValue[4] array, the sortplayer[4] array also follows the sorting steps of firsthandValue[4] array. So in the end, sortplayer[4] will get sorted according to the dice value.

Returns the Highest value rolled player

3.3 Game Loop

1. play[4] :
Playing order to left hand side rotation keeps in a array. Calls the next player through a loop and switch iteratively.
2. while() : game loops until one player bring its all pieces to Home

```
player.finished == 4
```

3.4 General Moves

For easier and more efficient way to control player behaviors is when there are no any specific behaviors act them under general behavior. To implement that there are general behavior that valid for all players.

3.4.1 Gets another roll

If player rolled 6 player gets another roll. But that player rolled 6 for three consecutive times, the roll get canceled and will pass to the next player by breaking a while loop that Runs player rolls 6. To track how many times the dice has rolled value six. That player stores the detail using

```
if (diceValue == 6 && player.sixrolled < 2)
{
    player.sixrolled++;
    {
        printf("Rolled 6 so player gets another roll\n");
        continue;
    }
}
else
{
    player.sixrolled = 0;
    break;
}
```

3.4.2 baseToStart

Player checks is there any pieces at base and if so the piece will bring to start position. After piece bring to the starting position. In base count, started Count, Piece location has to be updated and piece gets moving direction after a coin toss.

```
//increment board piece count
player->started += 1;
//decrement base piece count
player->inBase -= 1;
//piece position to start position
player->pieceData[i][2] = player->x;
//deciding the direction of the piece move
player->pieceData[i][4] = coinToss();
```

3.4.3 selectPlayPiece()

Player piece behaviour are implemented player specific way. If player do not have any prioritizing order the piece will selected as a cyclic manner like bluePlayer behaviors.

```

//give the player number Y-0,B-1,R-2,G-3
switch (player->playerIndex)
{
case 0:
    return yellowPlayPiece(diceValue);
    break;
case 1:
    return bluePlayPiece(diceValue);
    break;
case 2:
    return redPlayPiece();
    break;
case 3:
    return greenPlayPiece();
    break;
}

```

3.4.4 canCaptureAny()

After every move player checks whether capture happened or not. For that player has to iterate through opponent player pieces and find which pieces are in the same location as its pieces. If player has a catch again player gets another roll. Set of condition to make a capture as follows,

```

player != arrayplayerlist[j]    &&    // remove comparing same players pieces
player->pieceData[i][2] != YX    &&    // can not catch at starting position
player->pieceData[i][2] != BX    &&
player->pieceData[i][2] != RX    &&
player->pieceData[i][2] != GX    &&
player->pieceData[i][8] == -1    &&    //check if it in the Standardpath
player->pieceData[i][2] != BASE &&    // removing in base pieces
// if in the same block capture
player->pieceData[i][2] == arrayplayerlist[j]->pieceData[k][2])

```

To store capture data there is a integer size 5 array. Capture output will follow the captureData[5] array and it will always update after recognized that capture can be made.

```

captureData[0] //player;
captureData[1] //capturer piece;
captureData[2] //captured player;
captureData[3] //captured piece;
captureData[4] //captured location;

```

After every capture capturer and captured data will update, such as inBase pieces, Started pieces count,

```

//increment captured value of capturer

```

```

player->pieceData[captureData[1]][3]++;
//decremrnting captured pieces started count
player->inBase += 1;
//increment agian inBase count
player->started -= 1;
//captured pieces position to BASE
player->pieceData[captureData[3]][2] = BASE;
//captured pieces travelled value to zero
player->pieceData[captureData[3]][7] = 0;
//captured pieces captured count to zero
player->pieceData[captureData[3]][3] = 0;
player->pieceData[captureData[3]][8] = -1;

```

3.4.5 standardMove()

After selecting a piece according to the player behavior. The piece will move according to the dice value. In here move will go to clockwise or counter-clockwise. Player checks the piece direction and then make the move. IT is simple logic but has to satisfied requirmnet to move.

```

if clockwise
    (player->pieceDeatail[playpiece][2] + roll) % 52;
else counter-Clockwise
    player->pieceDeatail[playpiece][2] - roll;
    if (player->pieceDeatail[playpiece][2] <= 0)
    {
        51 + player->pieceDeatail[playpiece][2];
    }

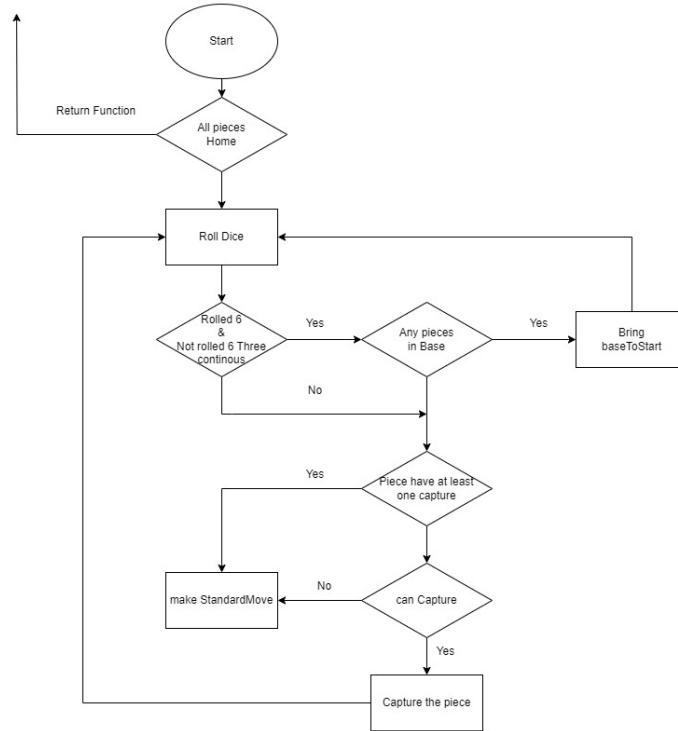
```


3.5 Player Behavior

3.5.1 Yellow Player

1. void yellowMove()

Player behaviour implemented using if else conditional Statements



2. yellowCatch()

Player Intentionally catch what it is required that only one capture but not more. Yellow player iterate using for loops to find out whether there any piece that can make a capture and it is not already captured opponent piece before. And yellow update the captureData[5] array

```
player1.pieceDeatail[i][3] == 0 //piece capture count must be zero
```

3. yellowPlayPiece()

Yellow player always moves the pieces that closest to its home

It has implemented by iterating through yellow pieces and choosing the closest unit that can enter to the Home path. for clockwise and counter-Clockwise direction method differ

```

if clockwise
    unitsToHome = 52 - player1.pieceDeatail[i][2];
if Counter-Clockwise
    if passed apporoch once
        unitsToHome = player1.pieceDeatail[i][2];
    if not passed apporch once
        unitsToHome = 51 + player1.pieceDeatail[i][2];

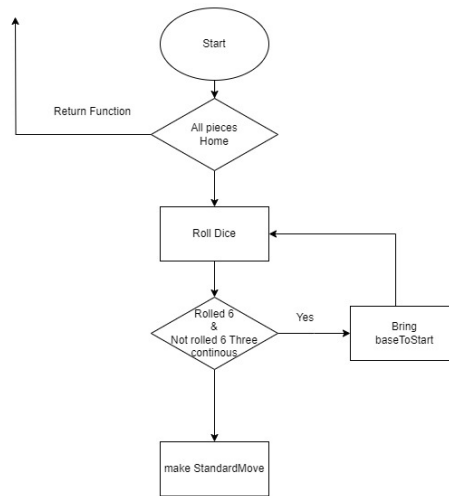
```

To find that the piece passed the apporoach or not `player1.pieceDeatail[i][4]` keeps the units it has travelled if `player1.pieceDeatail[i][4] > 2` it means piece has passed apporach cell

3.5.2 Blue Player

1. void blueMove()

Player behaviour implemented using if else conditional Statements



2. blueCatch()

Has not special conditions. Implemented using general capture behaviour.

3. bluePlayPiece()

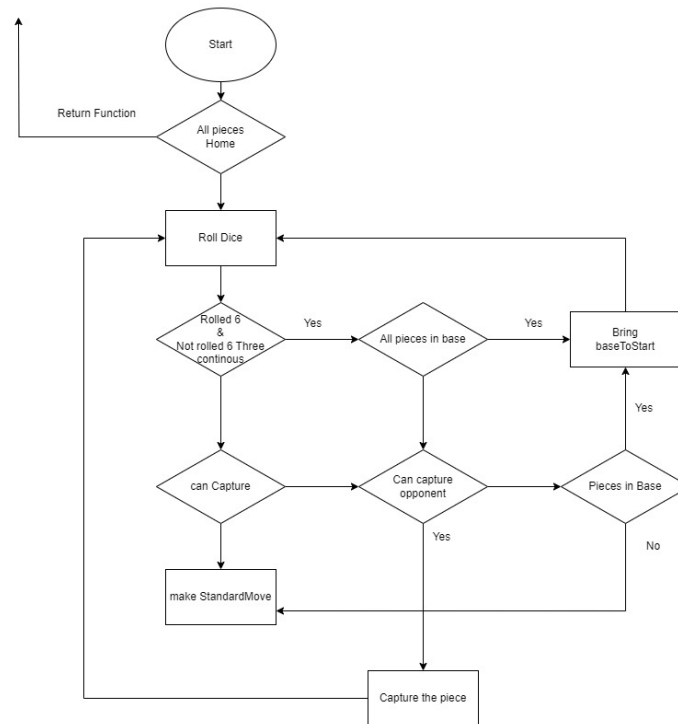
Blue player have cyclic manner. Piece is determined according to the **blueOrder**. Before every move blue player checks whether the piece according to the order can be move, if it is not; it moves the next blue piece and so on.

```
if (player2.pieceDeatail[blueOrder%4][2] != BASE &&
    player2.pieceDeatail[blueOrder%4][8] == -1)
{
    return blueOrder%4;
}
blueOrder = blueOrder+1;
```

3.5.3 Red Player

1. void redMove()

Player behaviour implemented using if else conditional Statements



2. redCatch()

Red player Catches the opponent piece that closes est to its approach cell. So Red player has to iterate through it self pieces and opponent pieces to find the which piece is closest to its approach cell.

```
mintoHome = player->o - player->pieceDeatail[k][2];
```

when Deciding Red players catch we have to iterate to every piece ensure that player selects the most suitable piece

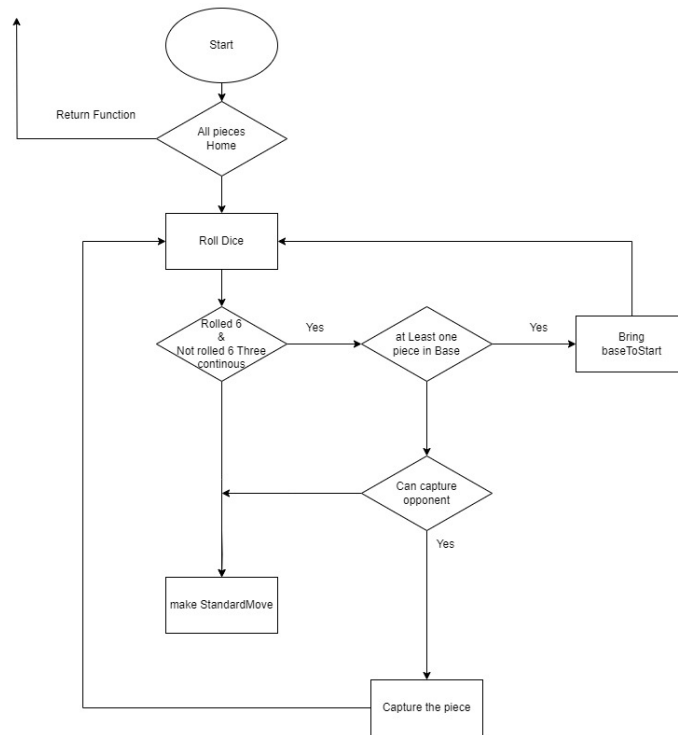
3. redPlayPiece()

Has not any specail implementations. Implemented as a cyclic manner like Blue player.

3.5.4 Green Player

1. void greenMove()

Player behaviour implemented using if else conditional Statements



2. greenCatch()

Green player also catch only what it required to enter home path. This implementation same as the yellow players capture conditions. Other capture conditions are specified in general capture condition part below.

```
//piece capture count must be zero
player4.pieceDeatail[i][3] == 0
```

when Deciding Red players catch we have to iterate to every piece ensure that player selects the most suitable piece

3. greenPlayPiece()

Because of Green player does not have any piece specific behavior. Green piece decide behavior implemented using cyclic manner as blue player

3.6 Finish

The first player who brings all pieces to its Home wins the game.

4 mistryCell

4.1 Generating

Mistry Cell will generate in Random locations from 0 to 51 cells. First mistry cell look can it be in that location if so it will be. If it can not it will loop until it find a random place that satisfy its requirments. The requirement implemneted as canBeMistry(),

1. check is there any pieces already
2. can not be the same cell as before

```
do
{
    mistryIs = rand() % 52;
    if(canBeMistry(mistryIs))
    {
        break;
    }
} while (0);

mistrycell = mistryIs;
```

4.2 Macros

```
#define BHAWANA 9
#define KOTUWA 27
#define PITAKOTUWA 46
#define ENERGIZE 2
#define SICK 0.5
#define FREEZE 0
#define FOURROUND 4
```

4.3 Teleport

When piece jumps to a mistry cell. Mistry cell sends the piece randomly to one of location given. To decide which location system will generate random value from 0 to 5 and jump by a switch statement according to that random value.

```
case 0: teleport to BHAWANA;
case 1: teleport to KOTUWA;
case 2: teleport to Pita KOTUWA;
```

```

case 3: teleport to BASE;
case 4: teleport to Start Cell of player;
case 5: teleport to Approach Cell of player;

```

4.4 Impacts

The impact of each Cell after the teleportation will record in pieceData[5] and pieceData[6] of each piece. If player detect that pieceData[5] got a value. He multiply the roll by that value. Using pieceData[5] can implement the Doubling, Halving and also Not moving effect for each piece individually. To count the effect for 4 rounds pieceData[6] keeps the data. It is decreasing one by one after each Round. After 4 round piece will behave as usual.

```

playpiece[5] = ENERGIZE; //multiply the dice roll by *2
playpiece[5] = Sick;      //multiply the dice roll by *0.5
playpiece[5] = FREEZE;    //multiply the dice roll by *0

```

5 Home Move

5.1 passApproch()

To enter home path piece should have satisfied some conditions that given below. Counterclockwise pieces passes the approach right after two cells because of that piece keeps how many cells it has travelled so far.

```

//can enter to the hoempath
//must travelled more than two sells
//at least onre capture has been made

```

To check a piece can enter to the home path. Player looks piece current position and after move position. If the approach cell is in between then player decide that piece can enter to the home path. The logic used to implement that is below. For yellow player has to implement it specially because yellow approach is right on cell 0;

```

if clockwise
    position before move < position after move
if counter clockwise
    position before move > position after move

```

If a piece satisfied above conditions and enter to the approach cell. The pieceData[8] = 0 that indicate piece now on the home path. By checking pieceData[8] loops and algorithms can remove pieces that not in the standard path.

5.2 homeMove()

If a dice value rolled a value and if it can add to a home path piece and the sum of roll and piece position is less than or equal 6 the dice value can be used to go

to the Home of player. Player should exactly rolled the dice same as cell count to finish the game by approaching Home.

```
if roled dice value less than cell count to home
    piece can move forward
else if dice value is equal to cell count
    piece wins the game
```

After each piece brought to the Home piece consider as wins. The first player who brings all pieces to Home wins the game.

6 Efficiency

6.1 Time

When considering time efficiency one of main problem is there are so many loops to iterate.

Some nested loops runs n^3 time. But in my perspective, I do not think it as a problem because the loops we iterate through are only have $4*4*4$ time complexity at worst case scenario. So i do not it as a problem because of the size of arrays are small, If they happened to be consider size large arrays. This looping algorithms will take much time to execute.

Using array data structure to keep data also increase time efficiency when it comes to finding and accessing data. So i think i this approach This is very effective way to implement the game.

6.2 Space

Considering space efficiency i have used structure to store player data and used arrays to store piece data. Even the data.

Because of using arrays it is efficient when accessing, deleting and inserting data. Variable space has minimummed using short data type for every numric value, Becouse in this game we will not be need a larger space to stor data, 16 bits will be enough.

Thank you